

Decorator pattern

1

1

You are going to open a star
fucks coffee chain

2

2

STARFUCKS



3

3

An example

Welcome to Starbuzz Coffee

Starbuzz Coffee has made a name for itself as the fastest growing coffee shop around. If you've seen one on your local corner, look across the street; you'll see another one.

Because they've grown so quickly, they're scrambling to update their ordering systems to match their beverage offerings.

When they first went into business they designed their classes like this...



4

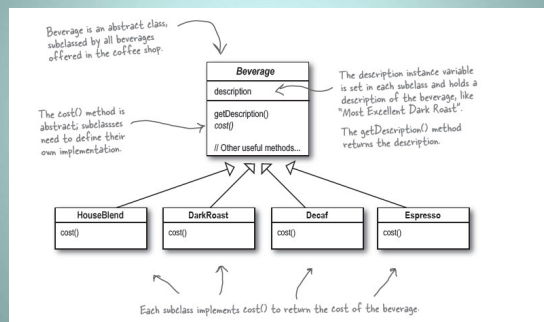
4

You are hired to implement an
table order system

5

5

Your first idea of implementation



6

6

[illegible]

Now a beverage can be mixed from different condiment to form a new beverage

This is stupid; why do we need all these classes? Can't we just use instance variables and inheritance in the superclass to keep track of the condiments?

Well, let's give it a try. Let's start with the Beverage base class and add instance variables to represent whether or not each beverage has milk, soy, mocha and whip...

Beverage
description
milk
soy
mocha
whip
getDescription() cost()
hasMilk() setMilk() hasSoy() setSoy() hasMocha() setMocha() hasWhip() setWhip()
// Other useful methods.

New boolean values for each condiment.

Now we'll implement cost() in Beverage (instead of keeping it abstract), so that it can calculate the costs associated with the condiments for a particular beverage instance. Subclasses will still override cost(), but they will also invoke the super version so that they can calculate the total cost of the base beverage plus the costs of the added condiments.

These get and set the boolean values for the condiments.

the decorator pattern

Now let's add in the subclasses, one for each beverage on the menu:

The superclass `cost()` will calculate the costs for all of the condiments, while the overridden `cost()` in the subclasses will extend that functionality to include costs for that specific beverage type.

Each `cost()` method needs to compute the cost of the beverage and then add in the condiments by calling the superclass implementation of `cost()`.

```
classDiagram
    class Beverage {
        description
        milk
        soy
        mocha
        whip
        +getDescription()
        +cost()
        +hasMilk()
        +setMilk()
        +hasSoy()
        +setSoy()
        +hasMocha()
        +setMocha()
        +hasWhip()
    }
    class HouseBlend {
        +cost()
    }
    class DarkRoast {
        +cost()
    }
    class Decaf {
        +cost()
    }
    class Espresso {
        +cost()
    }
    Beverage <|-- HouseBlend
    Beverage <|-- DarkRoast
    Beverage <|-- Decaf
    Beverage <|-- Espresso
```

Now, your turns. It is a good solution?

See, five classes total. This is definitely the way to go.

I'm not so sure; I can see some potential problems with this approach by thinking about how the design might need to change in the future.

12

Try several minutes to complete

Sharpen your pencil

Write the cost() methods for the following classes (pseudo-Java is okay):

```
public class Beverage {
    public double cost() {
```

```
public class DarkRoast extends Beverage {
    public DarkRoast() {
        description = "Most Excellent Dark Roast";
    }
    public double cost() {
```

you are here > 83

13

Some Hints

- Call parent class's method in an overridden method

```
class base {
    virtual void foo() { ... }
}

class son_of_base: base {
    virtual void foo() {
        base::foo(); // C++
        super.foo(); // java
        .... // do something else for your own
    }
}
```

14

What can you criticize about this inheritance architecture?

- Write down your notes to see if you are right

15

Sharpen your pencil

What requirements or other factors might change that will impact this design?

Price changes for condiments will force us to alter existing code.

New condiments will force us to add new methods and alter the cost method in the superclass.

We may have new beverages. For some of these beverages (iced tea?), the condiments may not be appropriate, yet the Tea subclass will still inherit methods like hasWhip().

What if a customer wants a double mocha?

As we saw in Chapter 1, this is a very bad idea!

Your turn:

16



Design Principle

Classes should be open for extension, but closed for modification.

17

Remember the old time



18

You open case to install the interface card



19

19

Some history about PC extension evolution

- In the past, an extension of your PC hardware means open your PC case with screw driver – which violate the design principle **close for modification**
- The design of open ISA, PCI, ATA... bus did fulfill the principle of **open for extension**.
- PC hardware has gone a long way to make extension easily
- USB come

20

20

Evil Apple ?



21

21



Come on in, we're open. Feel free to extend our classes with any new behavior you like. If your needs or requirements change (and we know they will), just go ahead and make your own extensions.



Sorry, we're closed. That's right, we spent a lot of time getting this code correct and bug free, so we can't let you alter the existing code. It must remain closed to modification. If you don't like it, you can speak to the manager.

22

22

Q: How can I make every part of my design follow the Open-Closed Principle?

A: Usually, you can't. Making OO design flexible and open to extension without the modification of existing code takes time and effort. In general, we don't have the luxury of tying down every part of our designs (and it would probably be wasteful). Following the Open-Closed Principle usually introduces new levels of abstraction, which adds complexity to our code. You want to concentrate on those areas that are most likely to change in your designs and apply the principles there.

Q: How do I know which areas of change are more important?

A: That is partly a matter of experience in designing OO systems and also a matter of the knowing the domain you are working in. Looking at other examples will help you learn to identify areas of change in your own designs.

23

23

Decorator Pattern

- The problems of two previous designs
 - we get class explosions, rigid designs, or we add functionality to the base class that isn't appropriate for some of the subclasses.

24

24

Revisit the problem again

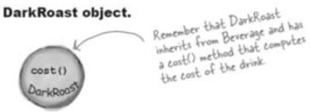
- If a customer wants a Dark Roast with Mocha and Whip
 - Take a DarkRoast object
 - Decorate it with a Mocha object
 - Decorate it with a Whip object
 - Call the cost() method and rely on delegation to add on the condiment costs

25

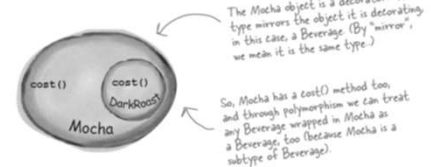
25

Constructing a drink order with Decorators

- 1 We start with our DarkRoast object.



- 2 The customer wants Mocha, so we create a Mocha object and wrap it around the DarkRoast.



26

26

The customer also wants Whip, so we create a Whip decorator and wrap Mocha with it.

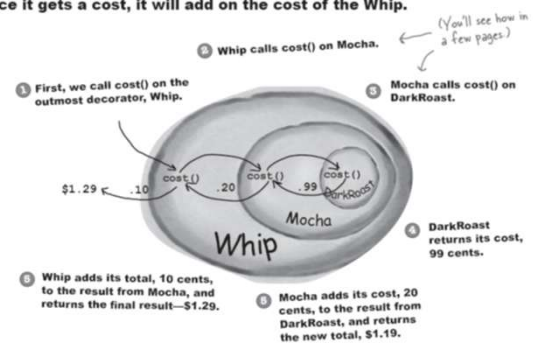


So, a DarkRoast wrapped in Mocha and Whip is still a Beverage and we can do anything with it we can do with a DarkRoast, including call its cost() method.

27

27

- 4 Now it's time to compute the cost for the customer. We do this by calling cost() on the outermost decorator, Whip, and Whip is going to delegate computing the cost to the objects it decorates. Once it gets a cost, it will add on the cost of the Whip.



28

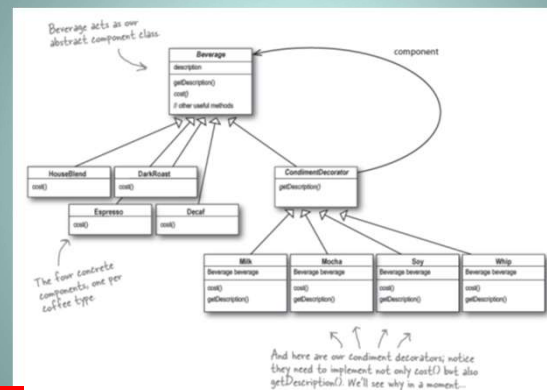
28

The Decorator Pattern attaches additional responsibilities to an object dynamically. Decorators provide a flexible alternative to subclassing for extending functionality.

29

29

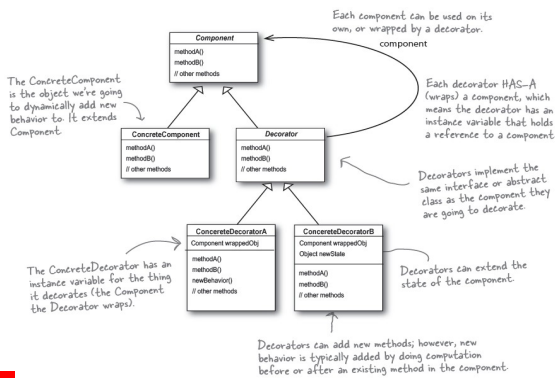
The decorator pattern for Starbuzz beverages



30

30

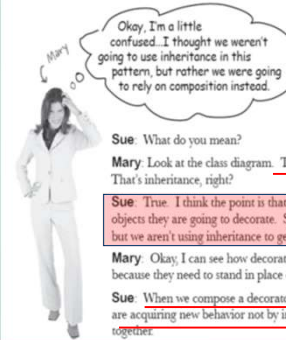
Decorator Pattern defined



31

31

Some confusion over Inheritance versus Composition



Sue: What do you mean?

Mary: Look at the class diagram. The CondimentDecorator is extending the Beverage class. That's inheritance, right?

Sue: True. I think the point is that it's vital that the decorators have the same type as the objects they are going to decorate. So here we're using inheritance to achieve the *type matching*, but we aren't using inheritance to get behavior.

Mary: Okay, I can see how decorators need the same "interface" as the components they wrap because they need to stand in place of the component. But where does the behavior come in?

Sue: When we compose a decorator with a component, we are adding new behavior. We are acquiring new behavior not by inheriting it from a superclass, but by composing objects together.

32

32

Mary: Okay, so we're subclassing the abstract class Beverage in order to have the correct type, not to inherit its behavior. The behavior comes in through the composition of decorators with the base components as well as other decorators.

Sue: That's right.

Mary: Oooh, I see. And because we are using object composition, we get a whole lot more flexibility about how to mix and match condiments and beverages. Very smooth.

Sue: Yes, if we rely on inheritance, then our behavior can only be determined statically at compile time. In other words, we get only whatever behavior the superclass gives us or that we override. With composition, we can mix and match decorators any way we like... *at runtime*.

Mary: And as I understand it, we can implement new decorators at any time to add new behavior. If we relied on inheritance, we'd have to go in and change existing code any time we wanted new behavior.

Sue: Exactly.

Mary: I just have one more question. If all we need to inherit is the type of the component, how come we didn't use an interface instead of an abstract class for the Beverage class?

Sue: Well, remember, when we got this code, Starbucks already had an abstract Beverage class. Traditionally the Decorator Pattern does specify an abstract component, but in Java, obviously, we could use an interface. But we always try to avoid altering existing code, so don't "fix" it if the abstract class will work just fine.

33

33

Let's see the code

```
public abstract class Beverage {
    String description = "Unknown Beverage";

    public String getDescription() {
        return description;
    }

    public abstract double cost();
}
```

Beverage is an abstract class with the two methods `getDescription()` and `cost()`.

`getDescription()` is already implemented for us, but we need to implement `cost()` in the subclasses.

34

34

The abstract class of condiments

```
public abstract class CondimentDecorator extends Beverage {
    public abstract String getDescription();
}
```

First, we need to be interchangeable with a Beverage, so we extend the Beverage class.

We're also going to require that the condiment decorators all implement the `getDescription()` method. Again, we'll see why in a sec...

35

35

Concrete Base Classes of Beverages

```
public class Espresso extends Beverage {
    public Espresso() {
        description = "Espresso";
    }

    public double cost() {
        return 1.99;
    }
}
```

Starbucks Coffee	
Coffees	
House Blend	.89
Dark Roast	.99
Decaf	1.05
Espresso	1.99
Condiments	
Steamed Milk	.10
Mocha	.15
Soy Whip	.10

we
re
instance
re.

Finally, we need to compute the cost of an Espresso. We don't need to worry about adding in condiments in this class, we just need to return the price of an Espresso: \$1.99.

36

36

A concrete Condiment class

Mocha is a decorator, so we extend `CondimentDecorator`.

Remember, `CondimentDecorator` extends `Beverage`.

We're going to instantiate Mocha with a reference to a `Beverage` using:

- (1) An instance variable to hold the beverage we are wrapping.
- (2) A way to set this instance variable to the object we are wrapping. Here, we're going to pass the beverage we're wrapping to the decorator's constructor.

We want our description to not only include the beverage – say "Dark Roast" – but also to include each item decorating the beverage, for example, "Mocha". So to the object we are decorating, then to that description.

Now we need to compute the cost of our beverage. First, we delegate the call to the object we're decorating, so that it can compute the cost; then, we add the cost of Mocha to the result.

When Mocha price changed, we only need to change this class.

```
public class Mocha extends CondimentDecorator {
    Beverage beverage;

    public Mocha(Beverage beverage) {
        this.beverage = beverage;
    }

    public String getDescription() {
        return beverage.getDescription() + ", Mocha";
    }

    public double cost() {
        return .20 + beverage.cost();
    }
}
```

37

37

Constructing new beverages from decorator classes dynamically

```
public class StarbuzzCoffee {

    public static void main(String args[]) {
        Beverage beverage = new Espresso();
        System.out.println(beverage.getDescription()
            + " $" + beverage.cost());

        Beverage beverage2 = new DarkRoast();
        beverage2 = new Mocha(beverage2);
        beverage2 = new Mocha(beverage2);
        beverage2 = new Whip(beverage2);
        System.out.println(beverage2.getDescription()
            + " $" + beverage2.cost());

        Beverage beverage3 = new HouseBlend();
        beverage3 = new Soy(beverage3);
        beverage3 = new Mocha(beverage3);
        beverage3 = new Whip(beverage3);
        System.out.println(beverage3.getDescription()
            + " $" + beverage3.cost());
    }
}
```

Order up an espresso, no condiments and print its description and cost.

Make a DarkRoast object.

Wrap it with a Mocha.

Wrap it in a second Mocha.

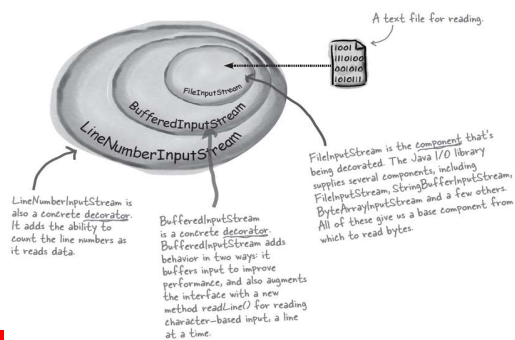
Wrap it in a Whip.

Finally, give us a HouseBlend with Soy, Mocha, and Whip.

38

38

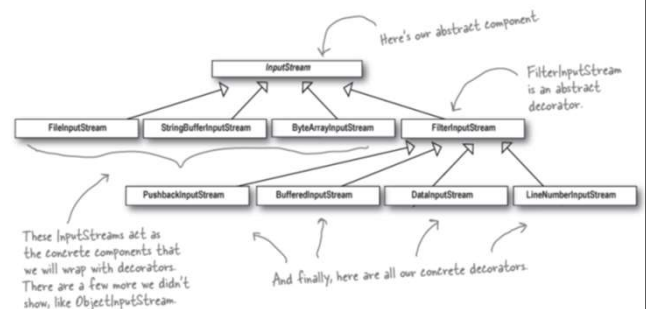
Real world decorator – Java I/O



39

39

Decorating the java.io classes



40

40

Comments

- You can see that this isn't so different from the Starbuzz design. You should now be in a good position to look over the java.io API docs and compose decorator s on the various *input* streams.
- You'll see that the *output* streams have the same design. And you've probably already found that the reader/writer streams (for character-based data) closely mirror the design of the streams classes (with a few differences and inconsistencies, but close enough to figure out what's going on).

41

41

Let's write a new decorator

Don't forget to import java.io. (not shown)

First, extend the `FilterInputStream`, the abstract decorator for all `InputStream`s.

```
public class LowerCaseInputStream extends FilterInputStream {
    public LowerCaseInputStream(InputStream in) {
        super(in);
    }

    public int read() throws IOException {
        int c = super.read();
        return (c == -1 ? c : Character.toLowerCase((char)c));
    }

    public int read(byte[] b, int offset, int len) throws IOException {
        for (int i = offset; i < offset+result; i++) {
            b[i] = (byte)Character.toLowerCase((char)b[i]);
        }
        return result;
    }
}
```

Now we need to implement two read methods. They take a byte (or an array of bytes) and convert each byte (that represents a character) to lowercase if it's an uppercase character.

42

42

Test out your new Java I/O decorator

```
public class InputTest {
    public static void main(String[] args) throws IOException {
        int c;
        try {
            InputStream in =
                new LowerCaseInputStream(
                    new BufferedInputStream(
                        new FileInputStream("test.txt")));

            while((c = in.read()) >= 0) {
                System.out.print((char)c);
            }

            in.close();
        } catch (IOException e) {
            e.printStackTrace();
        }
    }
}
```

Set up the stream and decorator and then LowerCase

I know the Decorator pattern

Just use the stream to read characters until the end of file and print as we go.

test.txt

43

43

Dark Side

- You can usually insert decorators transparently and the client never has to know it's dealing with a decorator
- However, if you write some code is dependent on specific types -> Bad things happen
- Java library is notorious to be used badly by people who do not know decorator pattern

44

44

The right way

```
Beverage beverage2 = new DarkRoast();
beverage2 = new Mocha(beverage2);
beverage2 = new Mocha(beverage2);
beverage2 = new Whip(beverage2);
System.out.println(beverage2.getDescription()
    + " $" + beverage2.cost());
```



```
Beverage beverage2 = new DarkRoast();
beverage2 = new Mocha(beverage2);
beverage2 = new Mocha(beverage2);
Whip beverage3 = new Whip(beverage2);
System.out.println(beverage3.getDescription()
    + " $" + beverage2.cost());
```

The poor way

45

45

Exercise solutions

```
public class Beverage {
    // declare instance variables for milkCost,
    // soyCost, mochaCost, and whipCost, and
    // getters and setters for milk, soy, mocha
    // and whip.
    public float cost() {
        float condimentCost = 0.0;
        if (hasMilk()) {
            condimentCost += milkCost;
        }
        if (hasSoy()) {
            condimentCost += soyCost;
        }
        if (hasMocha()) {
            condimentCost += mochaCost;
        }
        if (hasWhip()) {
            condimentCost += whipCost;
        }
        return condimentCost;
    }
}

public class DarkRoast extends Beverage {
    public DarkRoast() {
        description = "Most Excellent Dark Roast";
    }
    public float cost() {
        return 1.99 + super.cost();
    }
}
```

46

46

47

47